

## Cập Nhật Firmware OTA Qua UART Với Dual-Bank Flash

### 1. Yêu cầu:

Sử dụng ESP-IDF để xây dựng hệ thống cập nhật firmware Over-The-Air (OTA) trên ESP32 theo giao thức tùy chỉnh qua UART: nhận file firmware nhị phân từ PC, ghi vào phân vùng OTA trên Flash, kiểm tra tính toàn vẹn bằng CRC32, và khởi động lại vào firmware mới.

### 2. Tài liệu tham khảo:

- <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/ota.html>
- <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/uart.html>
- [https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/nvs\\_flash.html](https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/nvs_flash.html)
- [https://en.wikipedia.org/wiki/Cyclic\\_redundancy\\_check](https://en.wikipedia.org/wiki/Cyclic_redundancy_check)

### 3. Giải thích:

Cập nhật firmware Over-The-Air (OTA) là tính năng quan trọng của các thiết bị IoT đã triển khai thực tế. Tính năng này cho phép cài đặt firmware mới mà không cần tiếp cận vật lý với thiết bị. Trong project này, bản cập nhật được truyền qua UART thay vì Wi-Fi, mô phỏng kịch bản cập nhật tại hiện trường qua giao diện serial.

Flash của ESP32 được phân vùng thành hai slot ứng dụng: ota\_0 và ota\_1. Firmware đang chạy chiếm một slot. Bộ cập nhật OTA ghi firmware mới vào slot còn lại (slot không hoạt động). Sau khi xác minh thành công, bootloader được chỉ định boot từ slot mới trong lần khởi động tiếp theo.

Giao thức truyền firmware là giao thức tùy chỉnh dựa trên gói tin (packet-based). Mỗi gói tin có cấu trúc: [HEADER: 2 byte magic] [SEQ: 2 byte số thứ tự] [LEN: 2 byte độ dài payload] [PAYLOAD: tối đa 256 byte] [CRC16: 2 byte checksum của header+seq+len+payload]. PC host gửi các gói tin tuần tự. ESP32 phản hồi ACK hoặc NACK sau mỗi gói.

Quá trình OTA bắt đầu khi ESP32 nhận lệnh START\_OTA đặc biệt qua UART. Lệnh này bao gồm tổng kích thước firmware và giá trị CRC32 của toàn bộ file nhị phân. Sau khi nhận và ghi xong tất cả các gói tin, ESP32 tính CRC32 của dữ liệu đã ghi và so sánh với giá trị kỳ vọng. Nếu khớp, phân vùng mới được đặt làm phân vùng boot và thiết bị reset.

Nếu firmware mới không boot được (phát hiện bởi watchdog hoặc self-test thất bại), bootloader phải rollback về firmware trước đó. Cơ chế rollback sử dụng OTA rollback API của ESP-IDF.

Phía PC host phải cung cấp một script Python để đọc file nhị phân và truyền đến ESP32 theo giao thức gói tin đã định nghĩa. Script phải xử lý việc truyền lại khi nhận NACK và hiển thị thanh tiến trình (progress bar).

NVS (Non-Volatile Storage) phải được dùng để lưu bộ đếm số lần cập nhật. Nếu thiết bị reset hơn 3 lần mà không xác nhận firmware mới là hợp lệ, hệ thống phải tự động rollback về firmware cũ.

#### 4. Kế hoạch thực hiện:

- Nghiên cứu ESP-IDF OTA API, cấu hình partition table, và NVS API; thiết lập partition table với hai slot OTA. (1 ngày)
- Thiết kế và triển khai giao thức truyền UART dựa trên gói tin: cấu trúc gói, tính CRC16, logic ACK/NACK. (2 ngày)
- Triển khai OTA writer: nhận gói tin, ghi payload vào phân vùng OTA bằng `esp_ota_write()`, xử lý lỗi ghi. (2 ngày)
- Triển khai xác minh CRC32 toàn bộ firmware đã ghi; chuyển phân vùng boot bằng `esp_ota_set_boot_partition()`. (1 ngày)
- Triển khai logic rollback sử dụng bộ đếm NVS và `esp_ota_mark_app_valid_cancel_rollback()`. (1 ngày)
- Viết script Python phía host để truyền firmware, hiển thị tiến trình và xử lý truyền lại khi NACK. (1 ngày)
- Kiểm tra end-to-end: flash firmware tốt, cập nhật lên phiên bản mới, xác minh rollback khi image bị lỗi. (1 ngày)

#### 5. Ràng buộc:

- Tách code thành các layer: APP, UART\_PROTO, OTA\_WRITER, CRC\_UTIL, NVS\_STORE.
- Giao thức gói tin phải triển khai CRC16 (đa thức 0x1021) để kiểm tra tính toàn vẹn từng gói; CRC32 để xác minh toàn bộ image.
- Ghi OTA phải dùng API chính thức của ESP-IDF: `esp_ota_begin` / `esp_ota_write` / `esp_ota_end`; không được ghi Flash trực tiếp.
- Rollback phải được kích hoạt tự động nếu bộ đếm số lần cập nhật trong NVS vượt quá 3 mà không có xác nhận hợp lệ.
- Script Python phía host phải hỗ trợ truyền lại tối đa 3 lần mỗi gói khi nhận NACK trước khi hủy bỏ.
- Nhận UART phải dùng interrupt-driven với FreeRTOS queue; không được polling thanh ghi trạng thái UART.
- Hệ thống phải in log trạng thái rõ ràng qua UART tại mỗi giai đoạn OTA: START, WRITING (kèm phần trăm), VERIFY, SUCCESS/FAIL, REBOOT.
- Không được dùng biến global chia sẻ trực tiếp giữa các layer.

Vì phạm bất kỳ ràng buộc nào sẽ bị chấm 0 điểm.